# MAJEURE ES

# Embedded AI

**OLIVIER POTIN**
**MINES SAINT-ÉTIENNE**

**MINES Saint-Étienne**

**La culture de l'innovation au service du développement des entreprises**

INSPIRING INNOVATION SINCE 1816

MINES Saint-Étienne | INSPIRING INNOVATION SINCE 1816

# Embedded IA on MCU

## 12 hours of classes

1. General overview
2. Methodology for inference of NN with STM32CubeAI
3. MNIST example on STM32R9
4. Development of an Embedded NN Project with AIfES

MINES Saint-Étienne | INSPIRING INNOVATION SINCE 1816

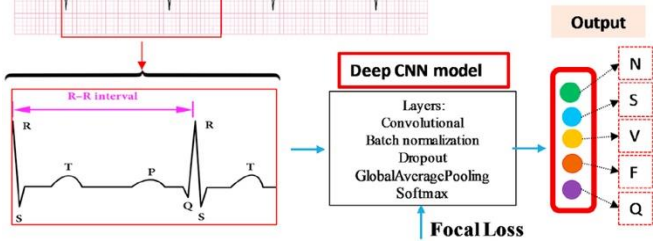
# Why ?

1. IA recognition (Computer Vision, Speech, …)
2. IA manufacturing (Condition monitoring, Failure prediction,…)
3. Classification (ECG…)
4. …

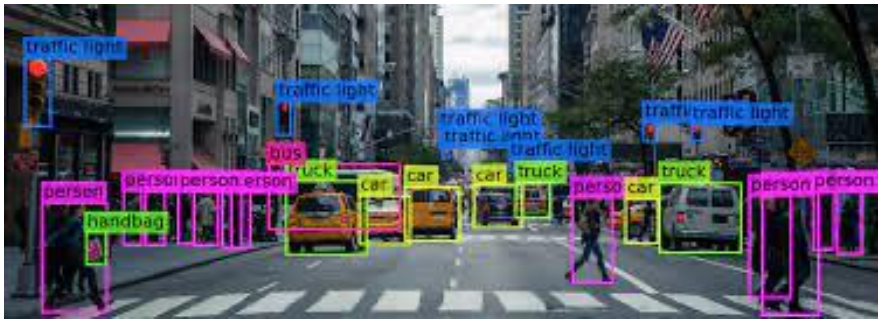
2.




3.




1.

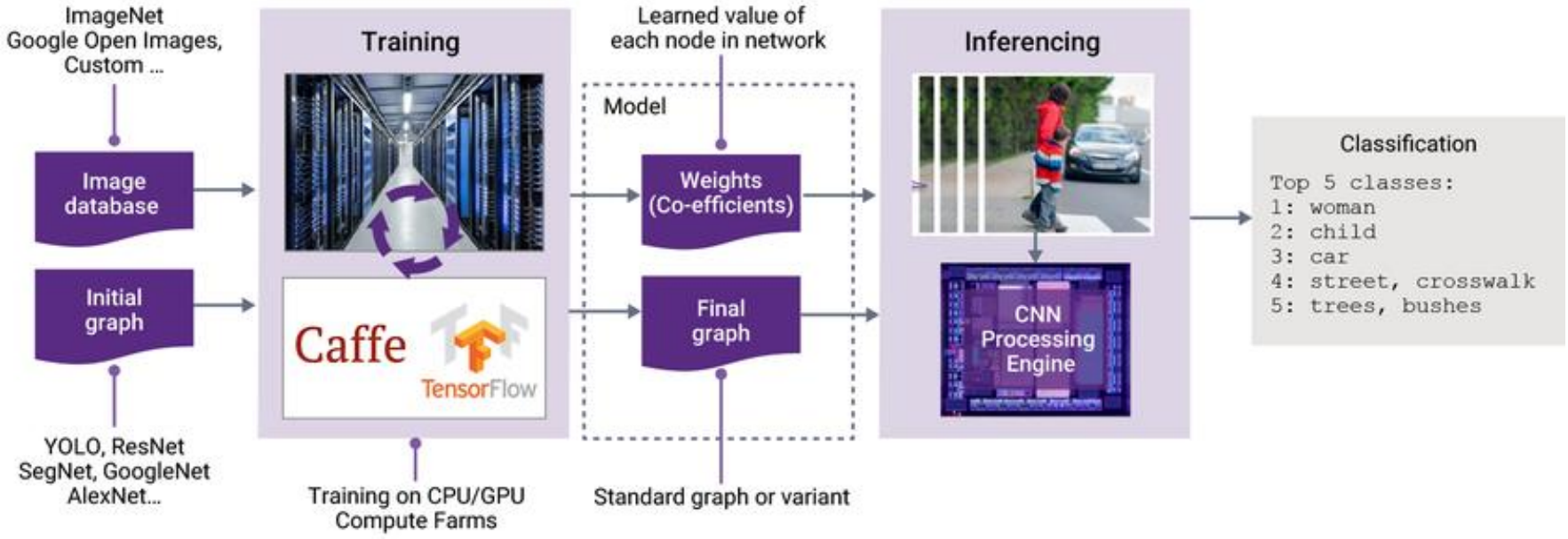


MINES Saint-Étienne | INSPIRING INNOVATION SINCE 1816

# Where ?

## Neural Network Architecture…

- Deploy ML on MCU for DNN inference

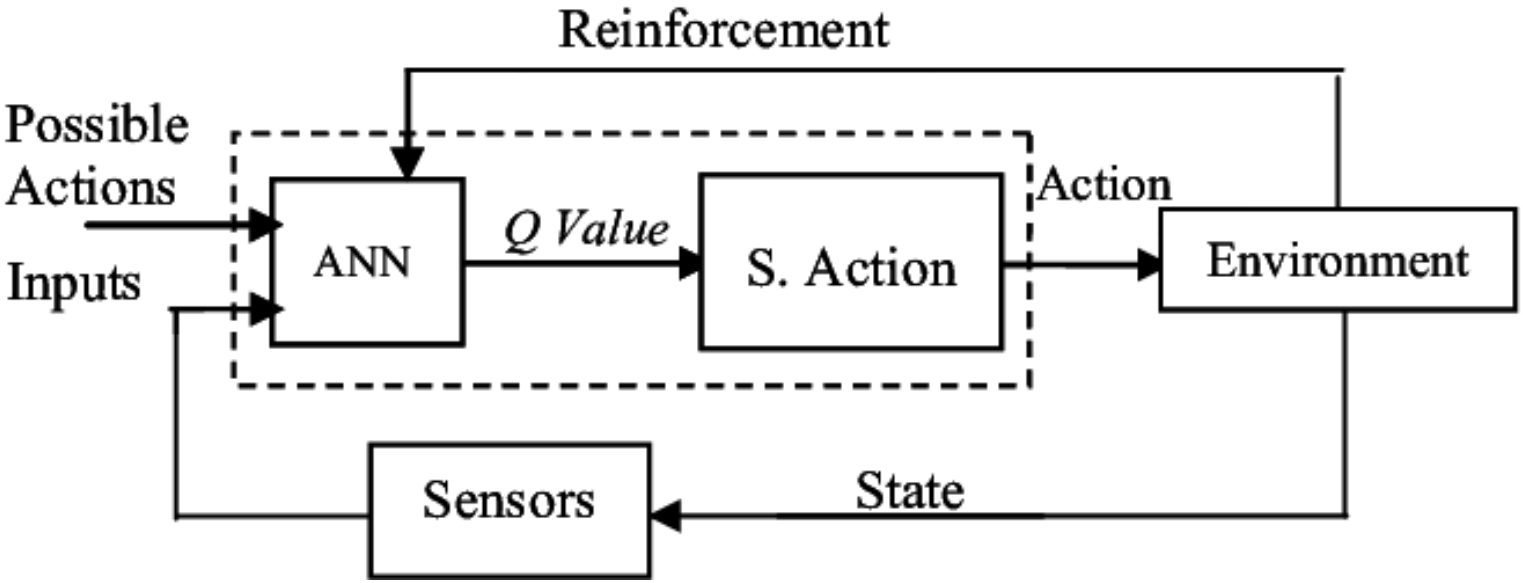


MINES Saint-Étienne | INSPIRING INNOVATION SINCE 1816

# Where ?

## Neural Network Architecture…

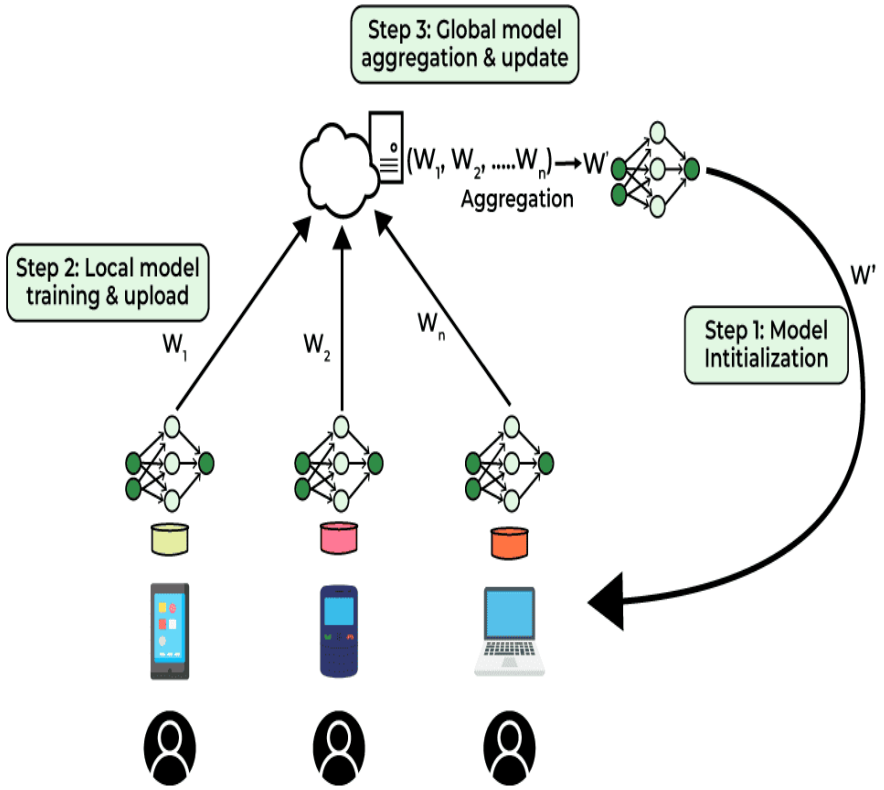
- Reinforcement learning
  - Example : NN from « https://huggingface.co » will reinforced with your own dataset



# Where ?

## Neural Network Architecture…

- Federated learning
  - Decentralizing processing to the network edge to bring it closer to the data to be processed
  - Optimized data processing as close as possible to the source
    - Minimize bandwidth between sensors and data processing centers
    - Ensure smooth and rapid response



MINES Saint-Étienne | INSPIRING INNOVATION SINCE 1816

# Where ?

## Neural Network Architecture…

- Federated learning (aka Benefits of not depending on the Cloud)



– Potential improvement in cybersecurity. The attackers must target multiple points to gain access to all data, rather than just the weakest point in the least secure device



– Energy efficiency : 90% of data sent to the Cloud is not reused
– Greater autonomy thanks to systems independent of the Cloud
(i.e. Complex decision-making without relying on the Cloud)



– Guaranteed uninterrupted operation (i.e. Secure operation)

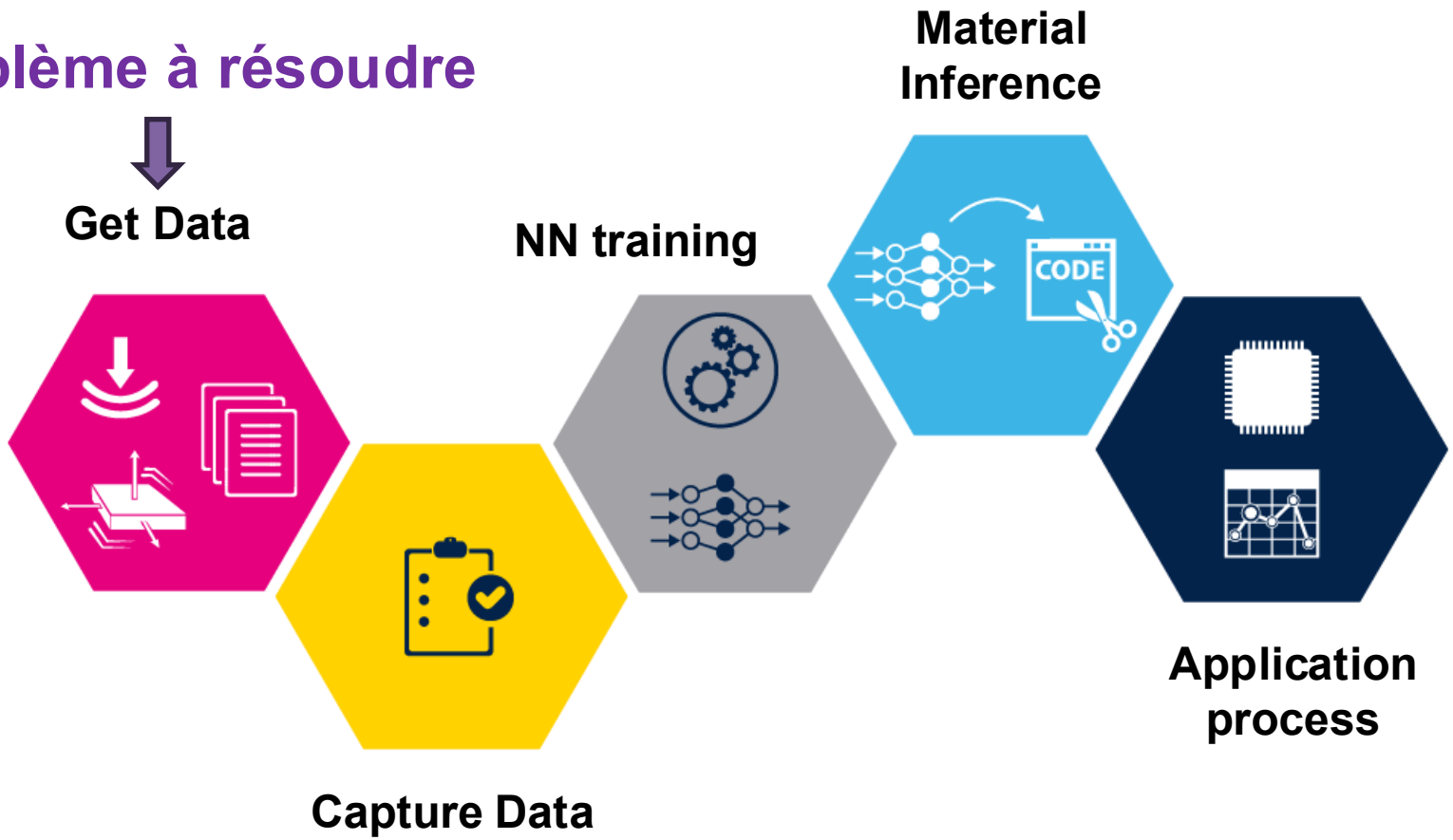


– Low latency
Round-trip distance to the Cloud exceeds 1,500 km: > 10 ms.
Embedding AI would reduce latency to ~1 ms or less



# Embedded DNN methodology

## Problème à résoudre




Courtesy ST Microelectronics

MINES Saint-Étienne | INSPIRING INNOVATION SINCE 1816

# Embedded DNN methodology

## Get data…



Capture a **sufficient** amount of **representative** data about the phenomenon that is being modeled.

This usually involves placing sensors on or near the object being monitored in order to record its state and changes over the time.

Examples of physical parameters include acceleration, temperature, sound, and visual depending on your application.

To resume : make a correct **analysis** of your dataset, **clean** your dataset of any outlier, **extract** features

Courtesy ST Microelectronics

MINES Saint-Étienne | INSPIRING INNOVATION SINCE 1816

# Embedded DNN methodology

## **Clean, label** data and build NN topology…



Creating an Artificial Neural Network (ANN) requires labeled data that has been acquired from sensors and pre-processing.

For so-called "supervised learning", the data sets must be characterized so that the different outputs can be classified correctly. This classified set is the "ground truth" that will be used to train the ANN and then validate it.

The developer must decide on the type of topology the ANN should have in order to best be able to learn from the data and provide useful output for the target application.

Courtesy ST Microelectronics

MINES Saint-Étienne | INSPIRING INNOVATION SINCE 1816

# Embedded DNN methodology

## Train the ANN…



**Training** the ANN involves passing the data sets through the Neural Network in an iterative manner so that the Network's outputs can **minimize desirable error criteria**.

ANN definition, training, and **testing** is typically performed using off-the-shelf Deep Learning frameworks. This is usually done on a powerful computing platform, with virtually unlimited memory and computational power, to allow many iterations in a short period of time.

The result of this training is the pre-trained Artificial Neural Network.

To resume : **Train**, **evaluate**, **optimize** and **test !!**

Keras

TensorFlow

Courtesy ST Microelectronics

MINES Saint-Étienne | INSPIRING INNOVATION SINCE 1816

# Train the DNN

## Prerequisites and Skills from the ML Course

Machine Learning Course (cf. https://ecampus.emse.fr/course/view.php?id=163)
And acquired skills

| |
|---|
| Understand the mechanisms of machine learning, particularly deep neural networks |
| Develop a machine learning model to perform a supervised classification task or an unsupervised clustering task |
| Apply optimization methods for neural network models to address overfitting issues |
| Analyze the results of a training process to detect issues related to data or algorithms |
| Evaluate the complexity of a machine learning task based on the available data, the complexity of the task to be accomplished, and state-of-the-art models |

# Train the DNN

## Choice of model architecture and training

⁉ The microcontroller imposes design limitations ⁉
ROM memory, RAM, clock frequency, floating-point unit, …



Convolutional Neural Network ?
Multi-Layer Perceptron ?

⁉ Computational capacity ⁉



Google COLAB (https://colab.research.google.com/)
Personal computer?
GPU ?
Use of a server ?

# Train the DNN

## Concerns about privacy...

⁉ **Source of data** ⁉

- *Private*
- *Public*

If the data are private, does it complies with GDPR ?

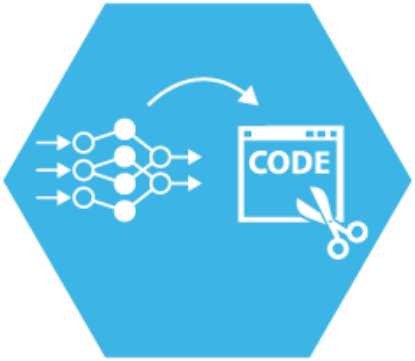
⁉ **Use of data** ⁉

- Military
- Health

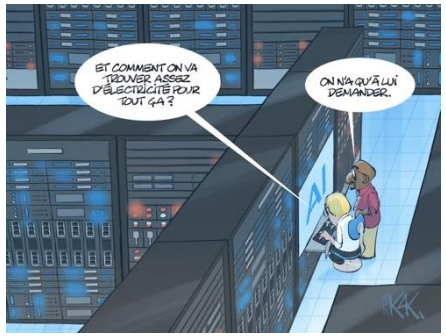
Authentication, Model integrity during inference,
Analysis of model vulnerabilities...

# Embedded DNN methodology

## Implement DNN : what are the constraints ?…




- Power consumption

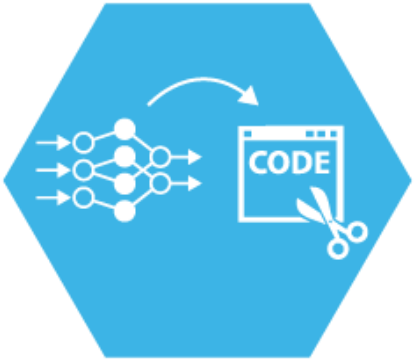



- Memory
  - ROM : store NN parameters & **program**
  - RAM : load/store intermediate calculation
- Clock frequency
  - Inference execution time

Courtesy ST Microelectronics

# Embedded DNN methodology

## Convert DNN into optimized code for MCU…

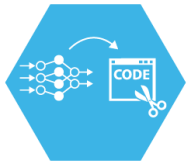



Embed the pre-trained DNN into a MCU which imply :

- optimizing code,
- minimizing complexity,
- respecting the memory requirements.

Courtesy ST Microelectronics

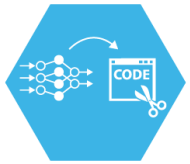
# Embedded DNN method




## Software tools to implement the DNN…

| Tool name | Bonus | Malus |
|---|---|---|
| STM32Cube.AI | fast, automatic conversion of pre-trained NNs into optimized code that can run on an STM32 | Pre-compiled libraries<br>Black box process |
| Link : https://www.st.com/en/embedded-software/x-cube-ai.html#overview | | |
| Lite Runtime (Lite RT) | IDE independent | Complexity<br>Black box |
| Link : https://ai.google.dev/edge/litert?hl=fr | | |

# Embedded DNN method

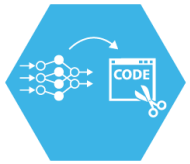



## Software tools to implement the DNN…

| Tool name | Bonus | Malus |
|---|---|---|
| Neural Network On Microcontroller (NNOM) | Open-source C files and librairies | Optimized ???<br>Quantization: 8 bits max |
| Link : https://github.com/majianjia/nnom | | |
| MCUNet: Tiny DL on IoT devices | Open-source files developped by MIT | Not improve for years |
| Link : https://hanlab.mit.edu/projects/mcunet | | |

MINES Saint-Étienne | INSPIRING INNOVATION SINCE 1816

# Embedded DNN method

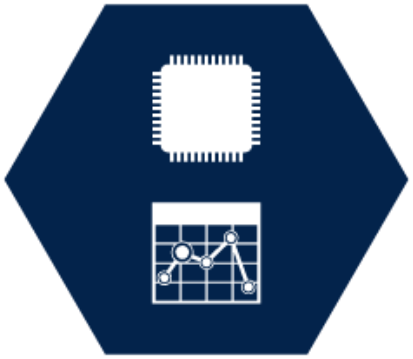



## Software tools to implement the DNN…

| Tool name | Bonus | Malus |
|---|---|---|
| AIfES (Fraunhofer) | Open-source C files and librairies | ??? |
| Link : https://fraunhofer-ims.github.io/AIfES_for_Arduino/ | | |
| AIDGE (CEA) | Open-source files | Peu de documentation, Support incomplet |
| Link : https://list.cea.fr/en/aidge/ | | |

# Embedded DNN method

## DNN implementation…



Finally, deploy the DNN in an MCU in your application.

***Software IDE***

Quickly prototype their innovative application thanks to integrated software packages

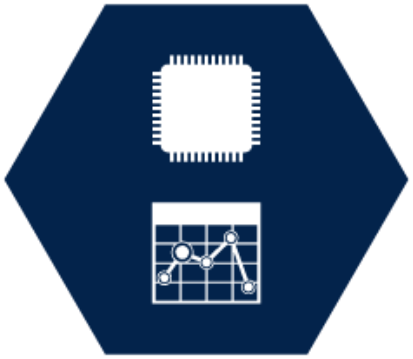
Courtesy ST Microelectronics

MINES Saint-Étienne | INSPIRING INNOVATION SINCE 1816

# Embedded DNN method

## DNN implementation…

Solutions classification :



| | CPUs | GPUs | Neural Network Hardware Accelerators |
|---|---|---|---|
| Ease of Development | Easy, pure SW, good tools, off-the-shelf IP | Easy, pure SW, good tools, off-the-shelf IP | Difficult, HW fixed at tapeout, SW must be partitioned between programmable core (CPU, GPU or DSP) and accelerator |
| Power Efficiency | Poor | Better than CPU, but still poor | Great for the offloaded layers, not all layers offloaded, adds significant data movement overhead |
| Future Proofing | Yes, always reprogrammable | Yes, always reprogrammable | No, high risk since as NNs evolve, current accelerator choices will become a poor fit for future NN styles |
| Max NN Performance per Core (TMAC/s) | <200GFLOP | ~200GFLOP | Up to 1TMAC |

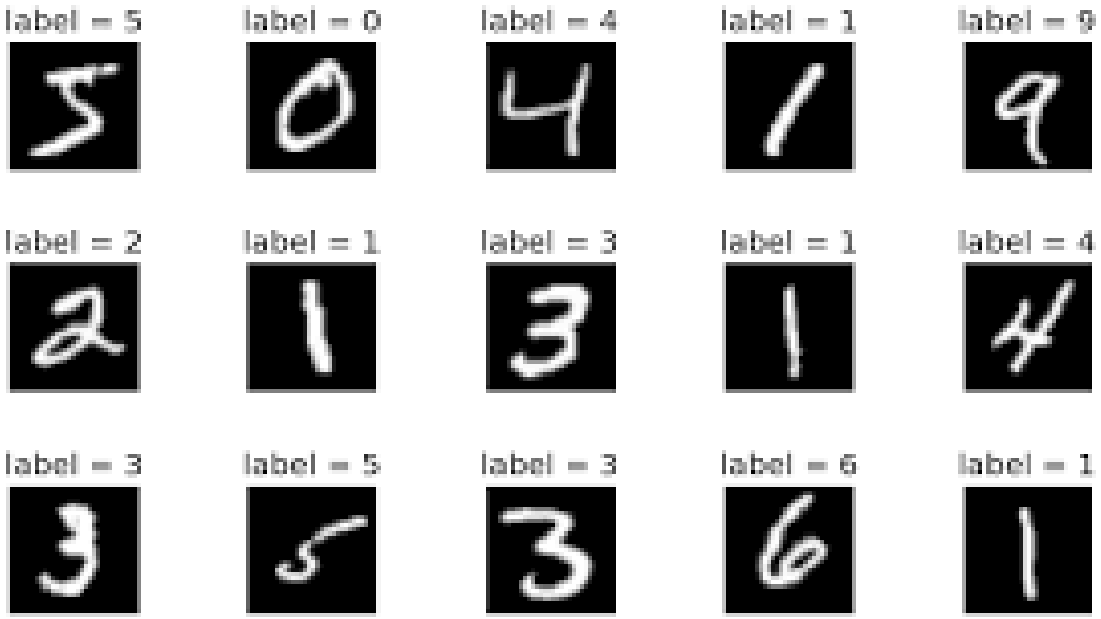
Courtesy ST Microelectronics

MINES Saint-Étienne | INSPIRING INNOVATION SINCE 1816

# Infering a DNN example

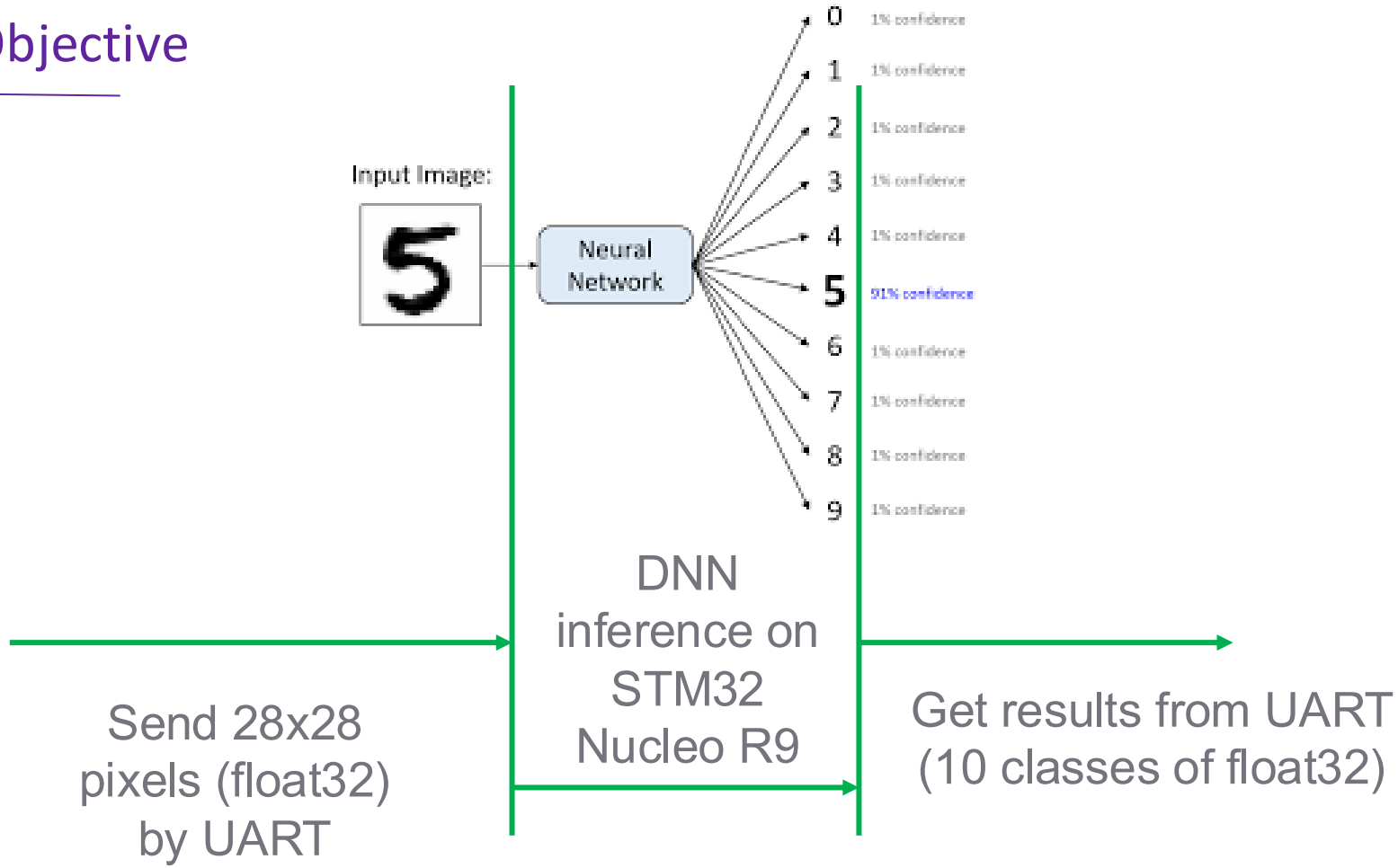
## The MNIST dataset

« The MNIST database is a large database of handwritten digits that is commonly used for training various image processing systems » (cf. Wikipedia)



# Infering a DNN example

## Objective



# Infering a DNN example

## Design and train a CNN

CNN → 784 inputs, « Conv2D + Relu + Softmax » layers → 10 outputs classes

```python
if __name__ == '__main__':

    save_files = True
    print("Start process ... \n")

    data = dataset()

    # Training of Neural Network
    trained_model = train_model(data)
    test_model(data, trained_model)

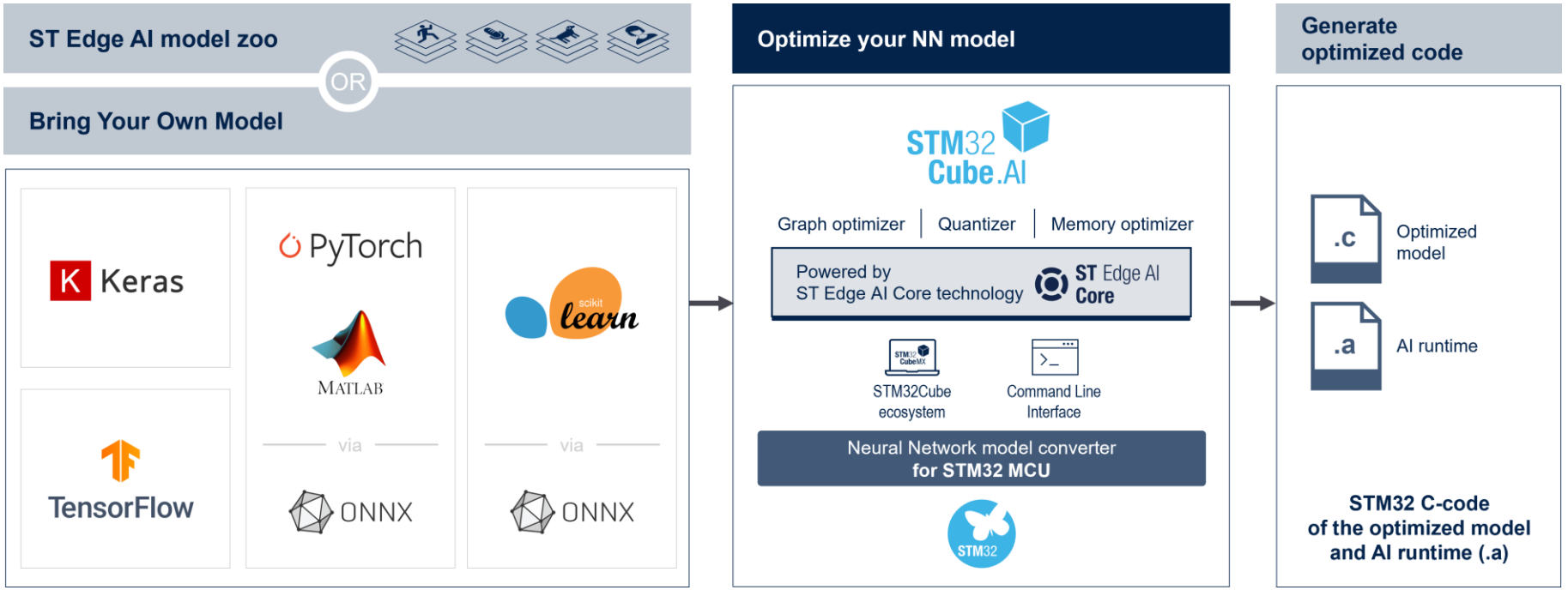
    # Saving model and testing datasets
    if save_files == True:
        np.save("MNIST_xtest_NN_C2_16_10.npy", data.x_test)
        np.save("MNIST_ytest_NN_C2_16_10.npy", data.y_test)
        trained_model.save("MNIST_NN_C2_16_10.h5")
```

Trained DNN saved as « .h5 » file

Keras

# Infering a DNN example

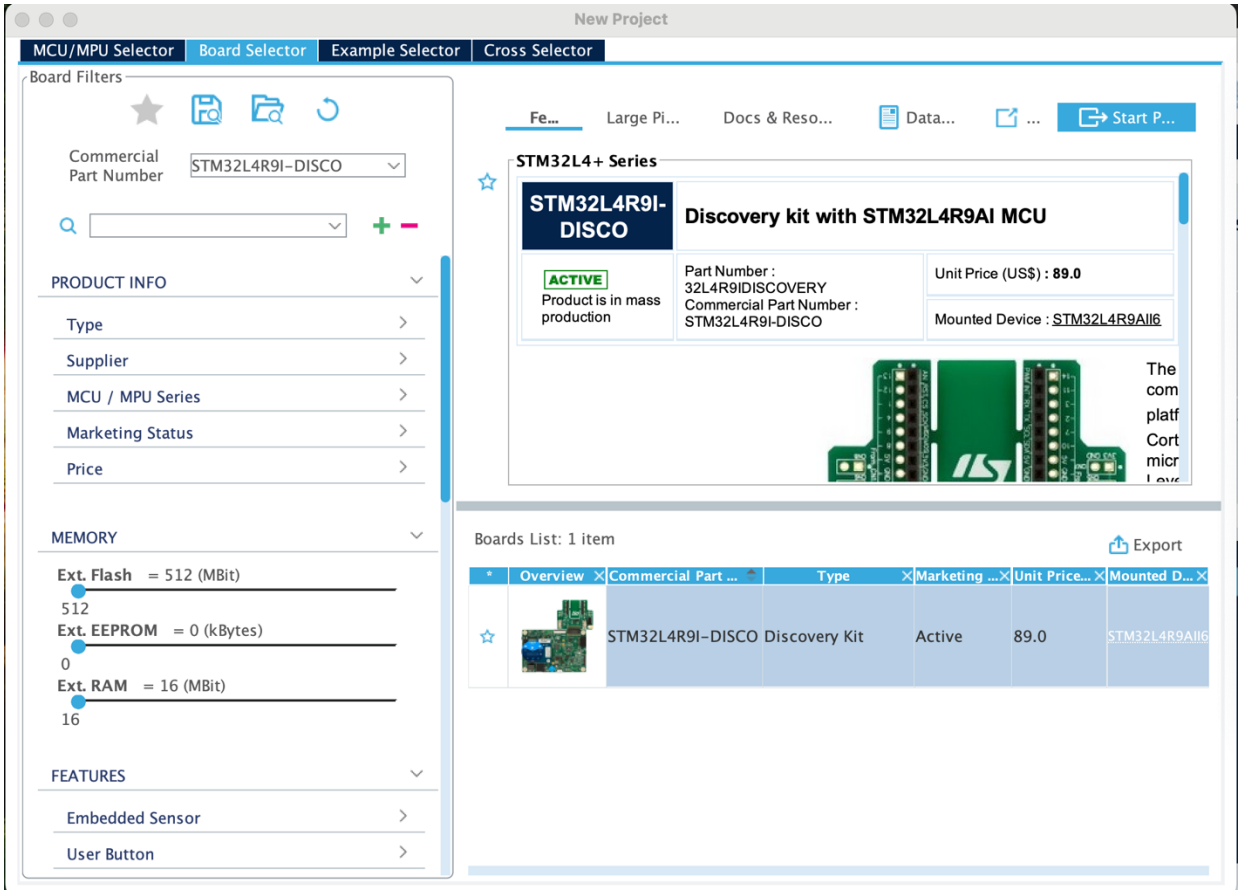
## The STM32 Cube MX – AI package




Courtesy ST Microelectronics

# Infering a DNN example

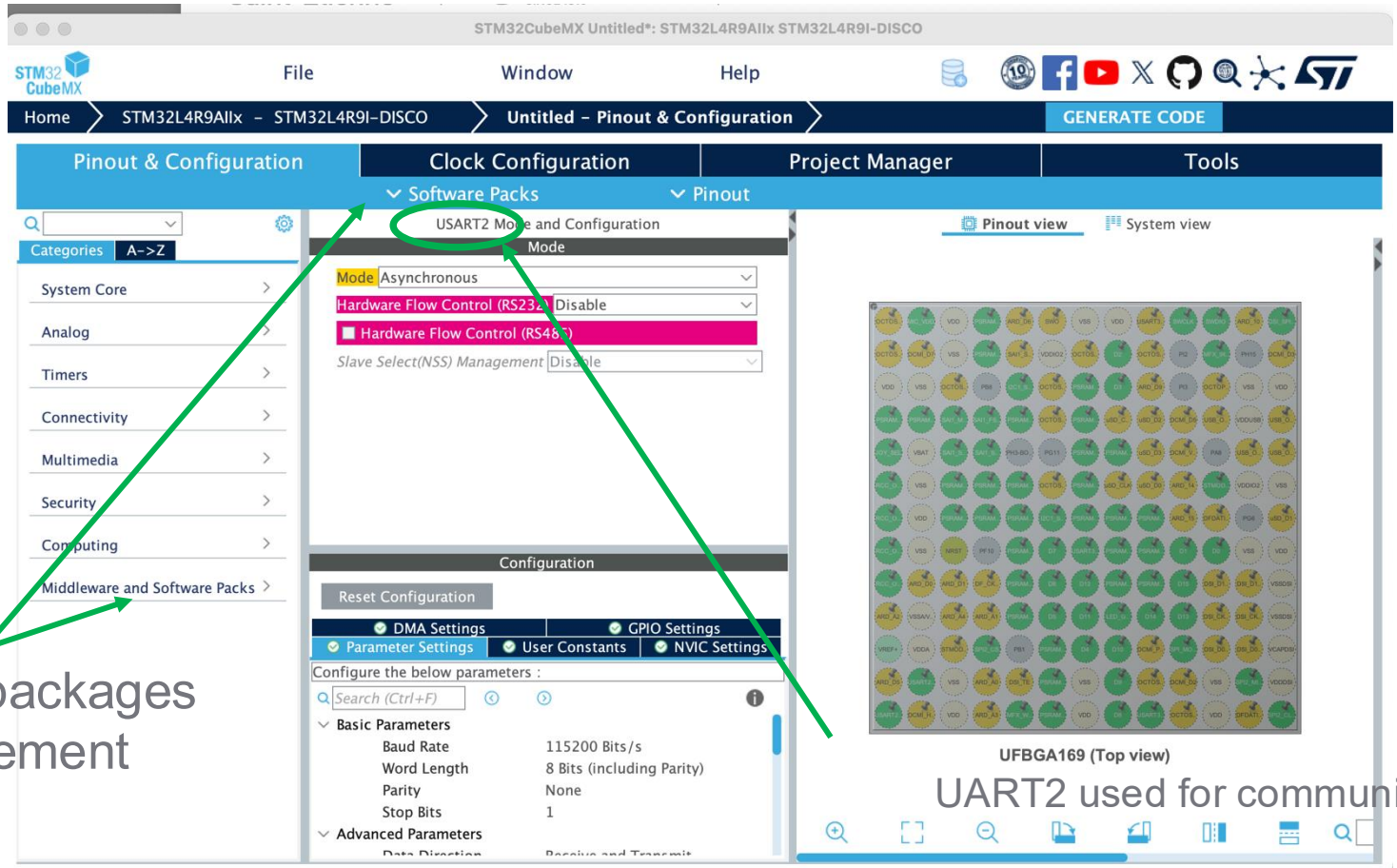
## The STM32 Cube MX – Create the project




Courtesy ST Microelectronics

# Infering a DNN example

## The CubeMX – AI package

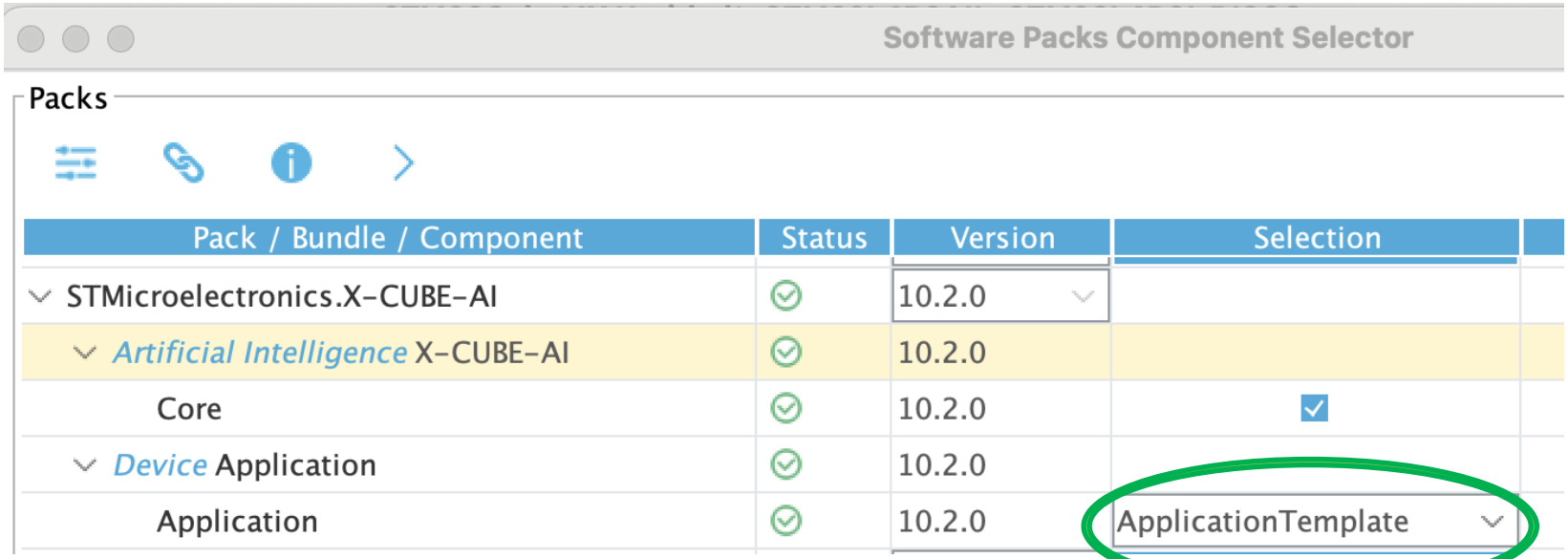



Software packages management

UART2 used for communication

# Infering a DNN example

## Choose the CubeMX – AI application



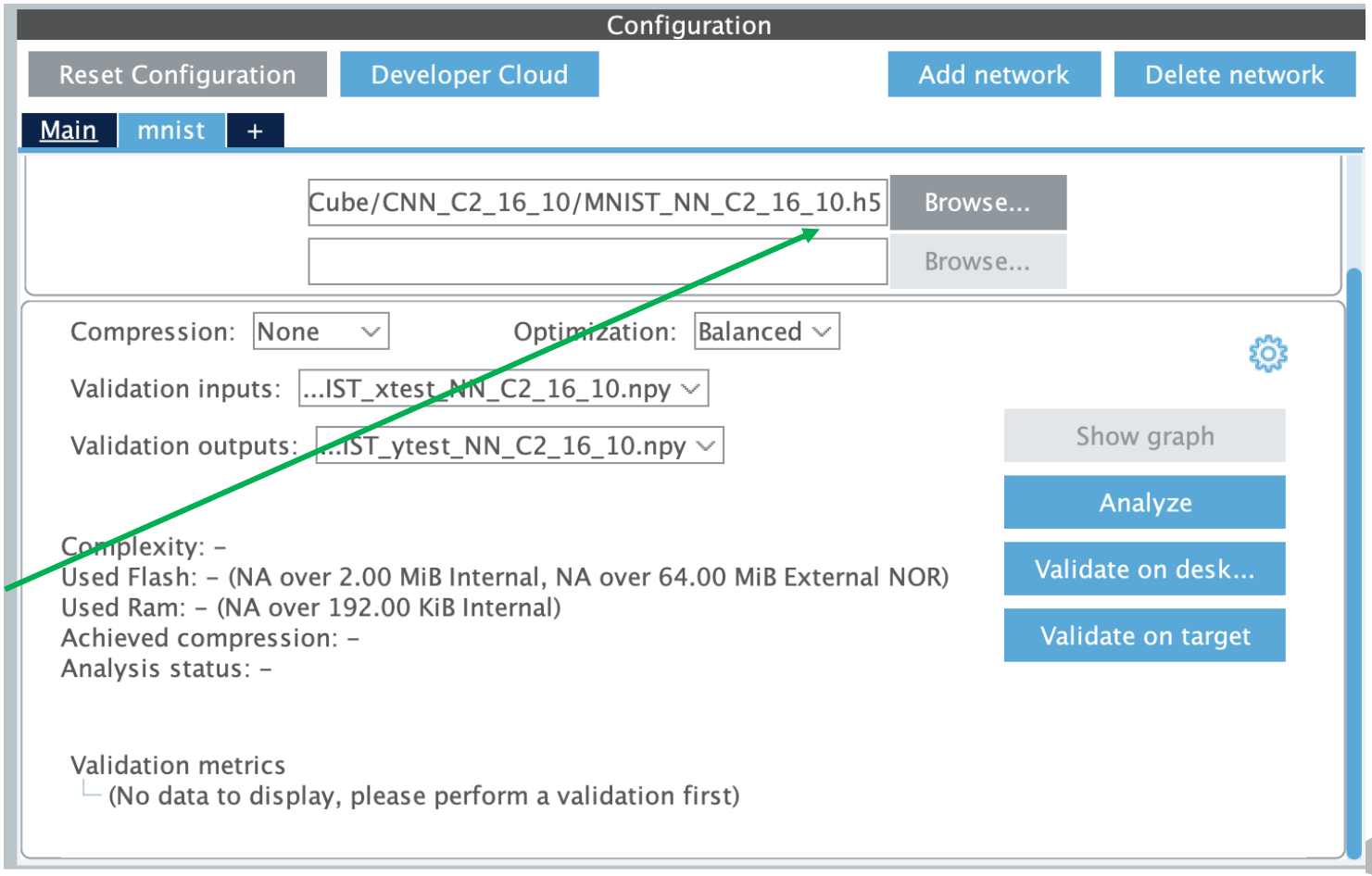

Software packages
Core libraries

- Performance
- Validation
- Application Template

# Infering a DNN example

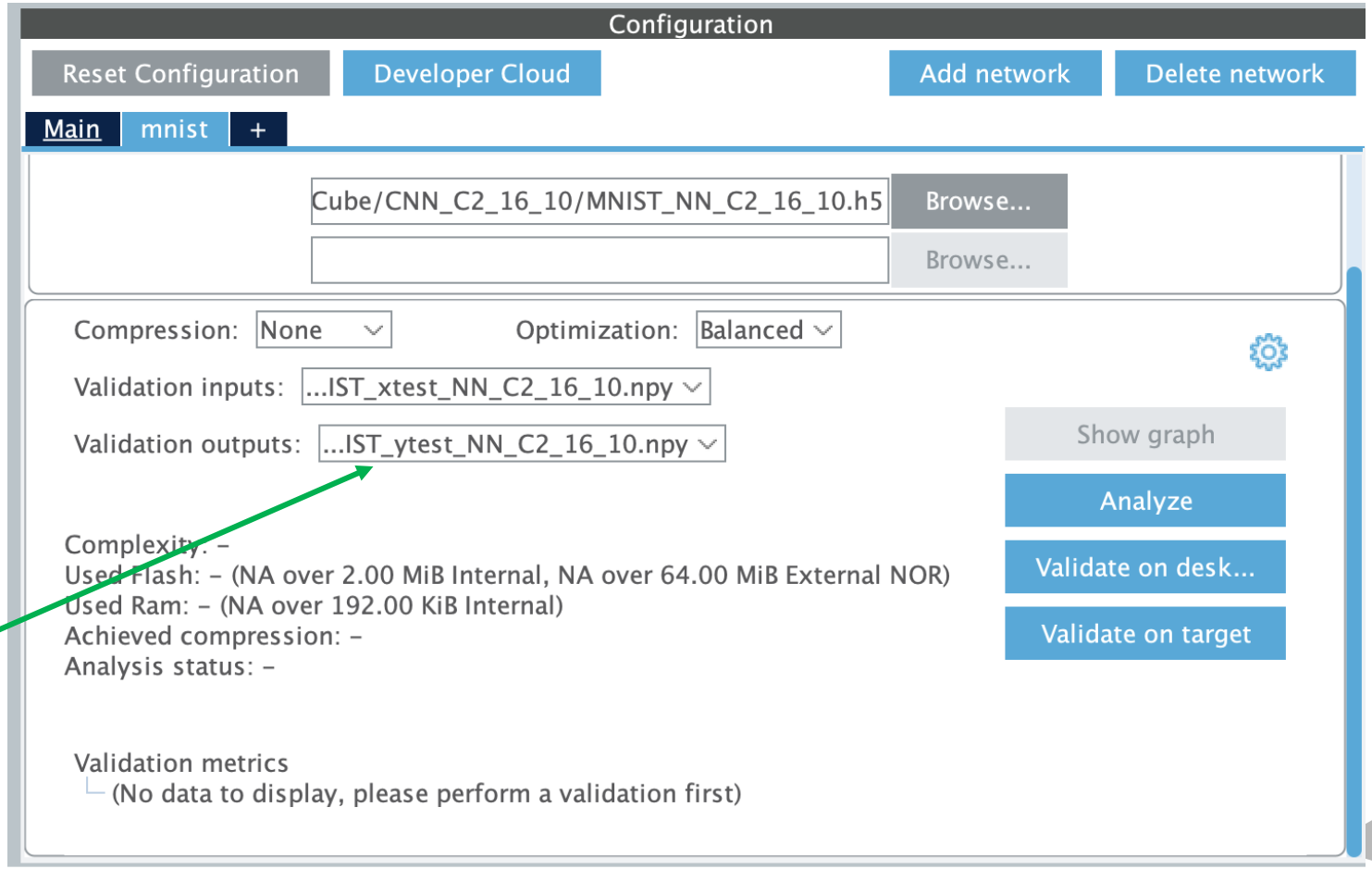
## Import the trained DNN in STM32L4R9

Set the « h5 » Keras model file



# Infering a DNN example

## Import the trained DNN in STM32L4R9

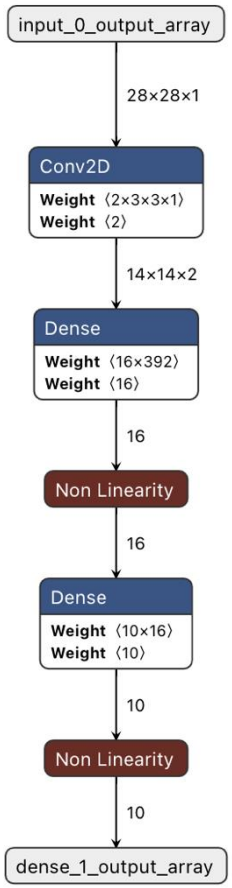



Define the validation dataset

# Infering a DNN example

## Analyze the trained DNN in STM32L4R9

### Imported NN graph




### NN analyze reports

Complexity: 23874 MACC
Used Flash: 25.30 KiB (25.30 KiB over 2.00 MiB Internal, .00 B over 64.00 MiB External NOR)
Used Ram: 3.78 KiB (3.78 KiB over 192.00 KiB Internal)
Achieved compression: -
Analysis status: done

Validation metrics

| | | |
|---|---|---|
| x86 c-model #1 | | |
| | Accuracy: | 96.64% |
| | MAE: | 0.00988 |
| | RMSE: | 0.07142 |
| | L2R: | 0.23142 |
| | Tensor name: | dense_1, ai_float, (1,1,1,10), m_id=[4] |
| original model #1 | | |
| | Accuracy: | 96.64% |
| | MAE: | 0.00988 |
| | RMSE: | 0.07142 |
| | L2R: | 0.23142 |
| | Tensor name: | dense_1, ai_float, (1,1,1,10), m_id=[4] |

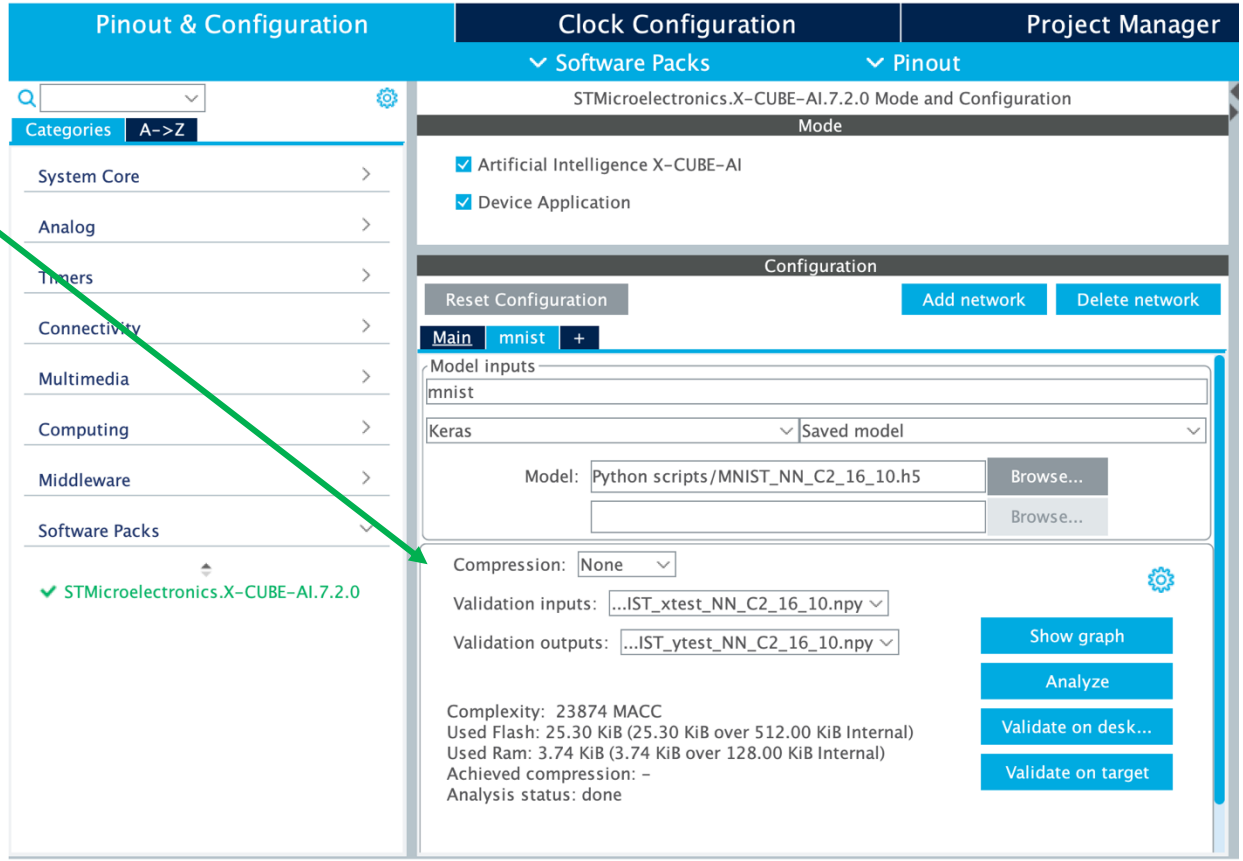
Is it too big ?

# Infering a DNN example

## Compress the trained DNN for a small STM

**Compression technics :**

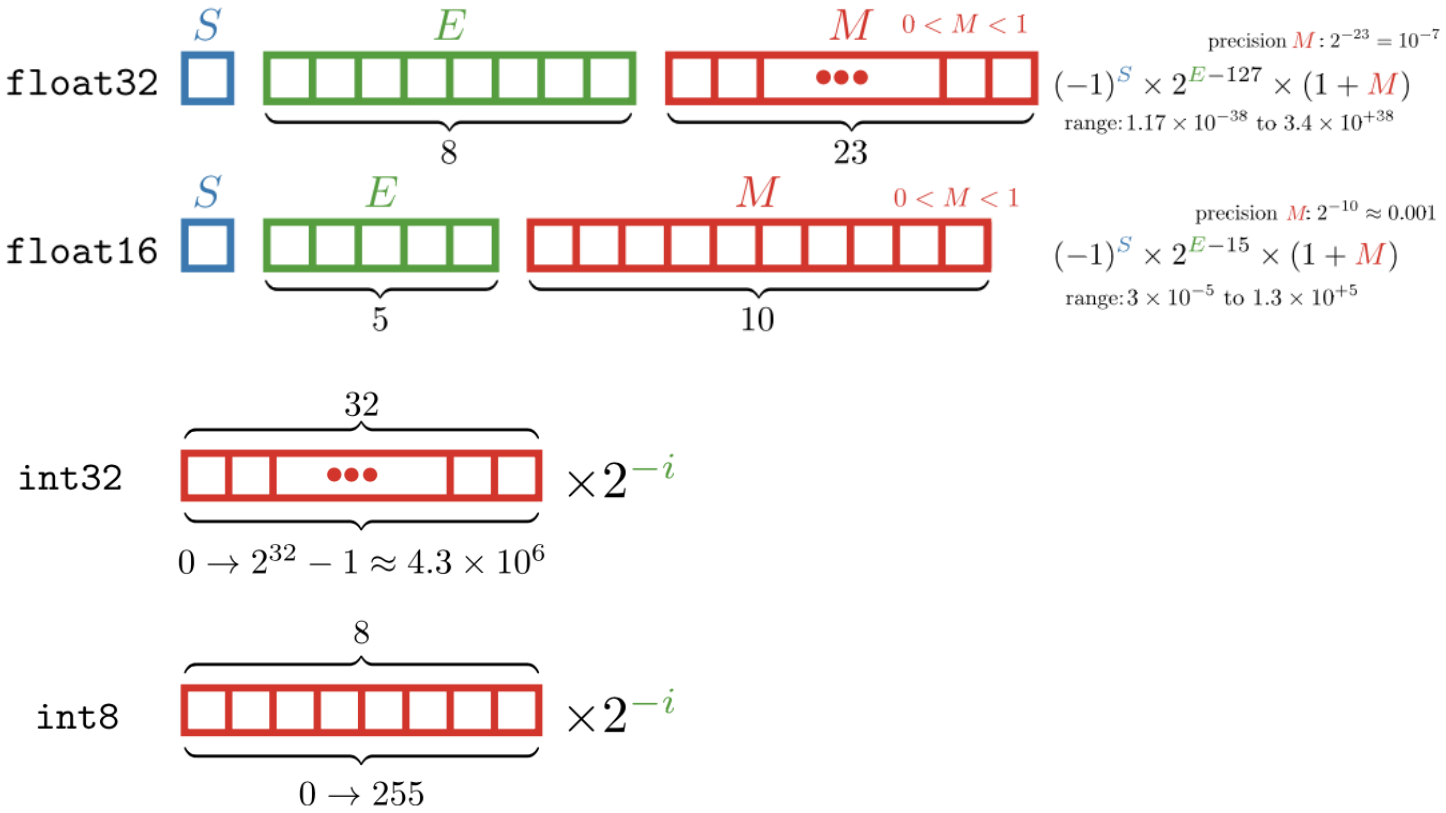
- Pruning
- Quantization
- LoRA



# Compress the DNN

## The « quantization »

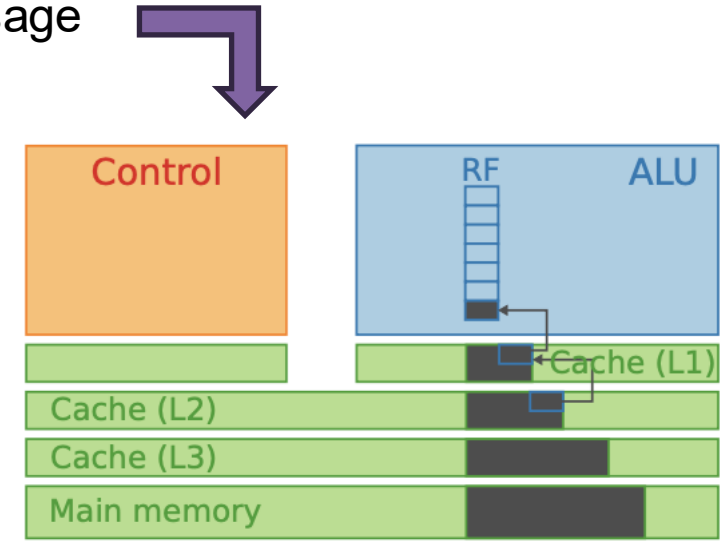
- Reduce model size
  - Fewer bits → Reduced memory footprint



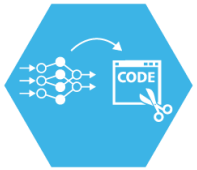
# Compress the DNN

## The « quantization »

- Reduce model size
  - Fewer bits → Reduced memory footprint
- Decrease memory access
  - GPU & CPU: reduce Cache usage



# Compress the DNN




## The « quantization »

- Reduce model size
  - Fewer bits → Reduced memory footprint
- Decrease memory access
  - GPU & CPU: reduce Cache usage
- Computational complexity

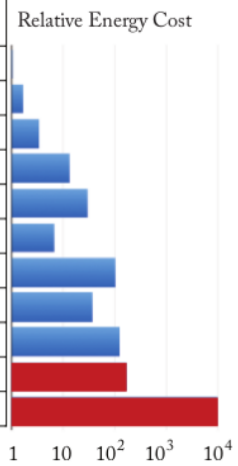
- *To add two FP numbers:*

Shift M according to E (int shift $n_E$ bits)
Add M (int add $n_M$ bits)
Normalize(0 <M <1)

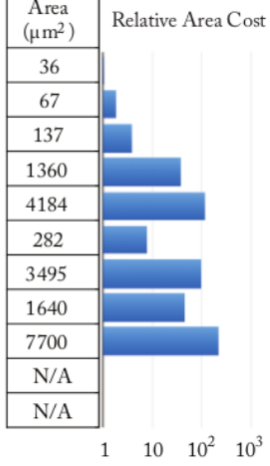
- *To multiply two FP numbers:*

Multiply M (int mult $n_M$ bits)
Add E (int add $n_E$ bits)
Normalize (0 <M <1)

| Operation: | Energy (pJ) |
|---|---|
| 8b Add | 0.03 |
| 16b Add | 0.05 |
| 32b Add | 0.1 |
| 16b FP Add | 0.4 |
| 32b FP Add | 0.9 |
| 8b Mult | 0.2 |
| 32b Mult | 3.1 |
| 16b FP Mult | 1.1 |
| 32b FP Mult | 3.7 |
| 32b SRAM Read (8KB) | 5 |
| 32b DRAM Read | 640 |

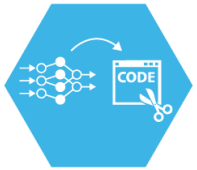



| Area ($\mu m^2$) |
|---|
| 36 |
| 67 |
| 137 |
| 1360 |
| 4184 |
| 282 |
| 3495 |
| 1640 |
| 7700 |
| N/A |
| N/A |



MINES Saint-Étienne | INSPIRING INNOVATION SINCE 1816

# Compress the trained DNN




## Quantization: convert float32 → int16, int8, binary

Complexity: 23874 MACC
Used Flash: 25.30 KiB (25.30 KiB over 512.00 KiB Internal)
Used Ram: 3.74 KiB (3.74 KiB over 128.00 KiB Internal)
Achieved compression: -
Analysis status: done

÷ 8

Complexity: 23874 MACC
Used Flash: 3.45 KiB (25.30 KiB over 512.00 KiB Internal)
Used Ram: 3.74 KiB (.00 B over 128.00 KiB Internal)
Achieved compression: 7.63
Analysis status: error

Validation metrics

x86 c-model #1

| | |
|---|---|
| Accuracy: | 96.64% |
| MAE: | 0.00988 |
| RMSE: | 0.07142 |
| L2R: | 0.23142 |
| Tensor name: | dense_1, ai_float, (1,1,1,10), m_id=[4] |

original model #1

| | |
|---|---|
| Accuracy: | 96.64% |
| MAE: | 0.00988 |
| RMSE: | 0.07142 |
| L2R: | 0.23142 |
| Tensor name: | dense_1, ai_float, (1,1,1,10), m_id=[4] |

-0,2%

Validation metrics

x86 c-model #1

| | |
|---|---|
| Accuracy: | 96.43% |
| MAE: | 0.01037 |
| RMSE: | 0.07283 |
| L2R: | 0.23635 |
| Tensor name: | dense_1, ai_float, (1,1,1,10), m_id=[4] |

original model #1

| | |
|---|---|
| Accuracy: | 96.64% |
| MAE: | 0.00988 |
| RMSE: | 0.07142 |
| L2R: | 0.23142 |
| Tensor name: | dense_1, ai_float, (1,1,1,10), m_id=[4] |

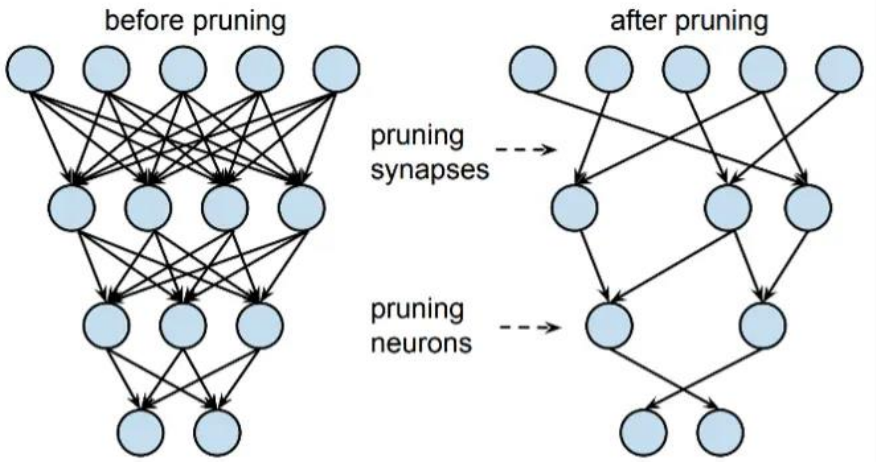
X-cross #1

| | |
|---|---|
| Accuracy: | 98.98% |
| MAE: | 0.00281 |
| RMSE: | 0.01848 |
| L2R: | 0.05997 |
| Tensor name: | dense_1, ai_float, (1,1,1,10), m_id=[4] |

# Compress the DNN

## The « Pruning »

Definition: Set some weights to zero or suppress useless neurons

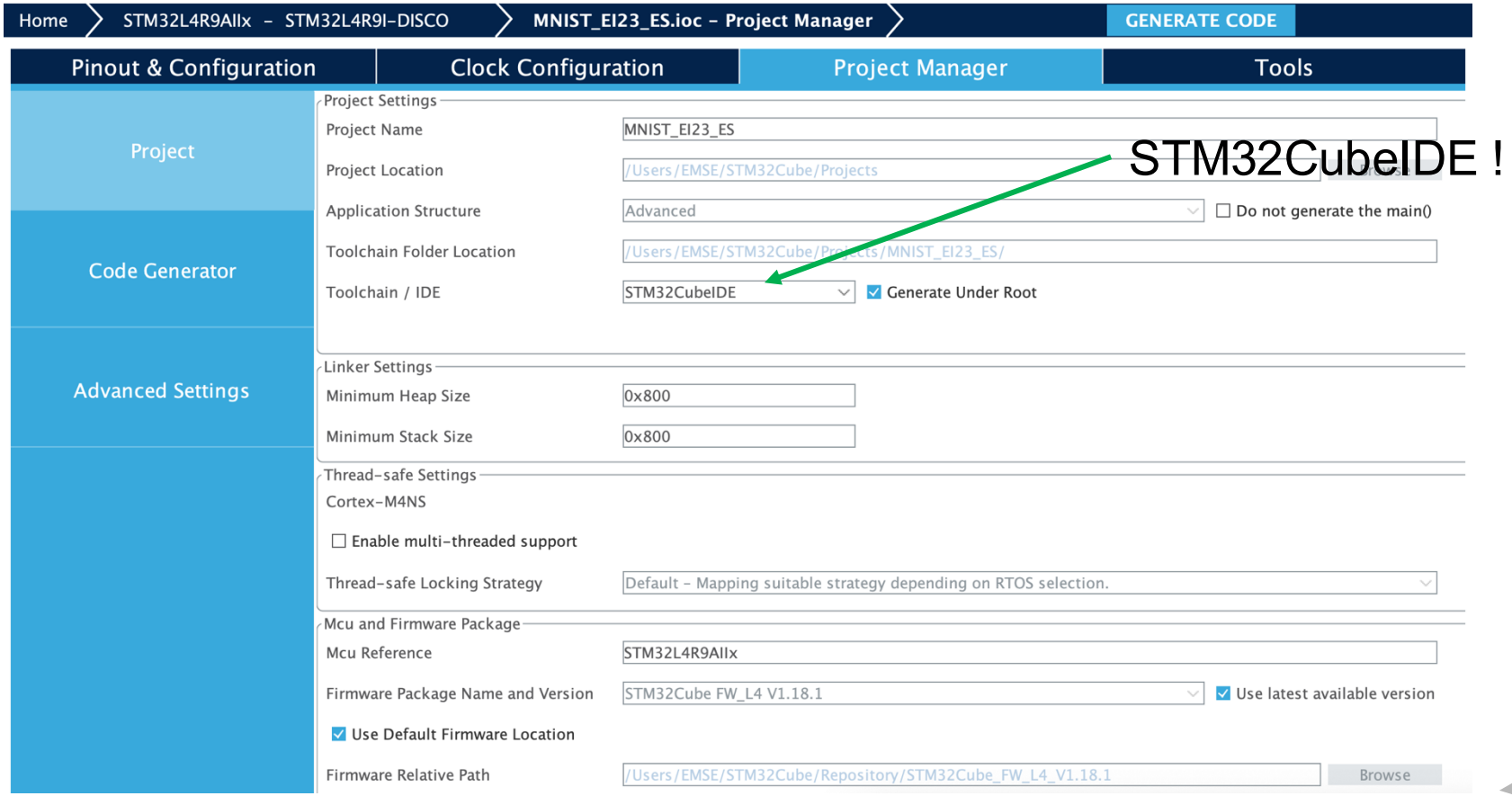



Keras : *pruning_low_magnitude API*

TensorFlow : projet MorphNet

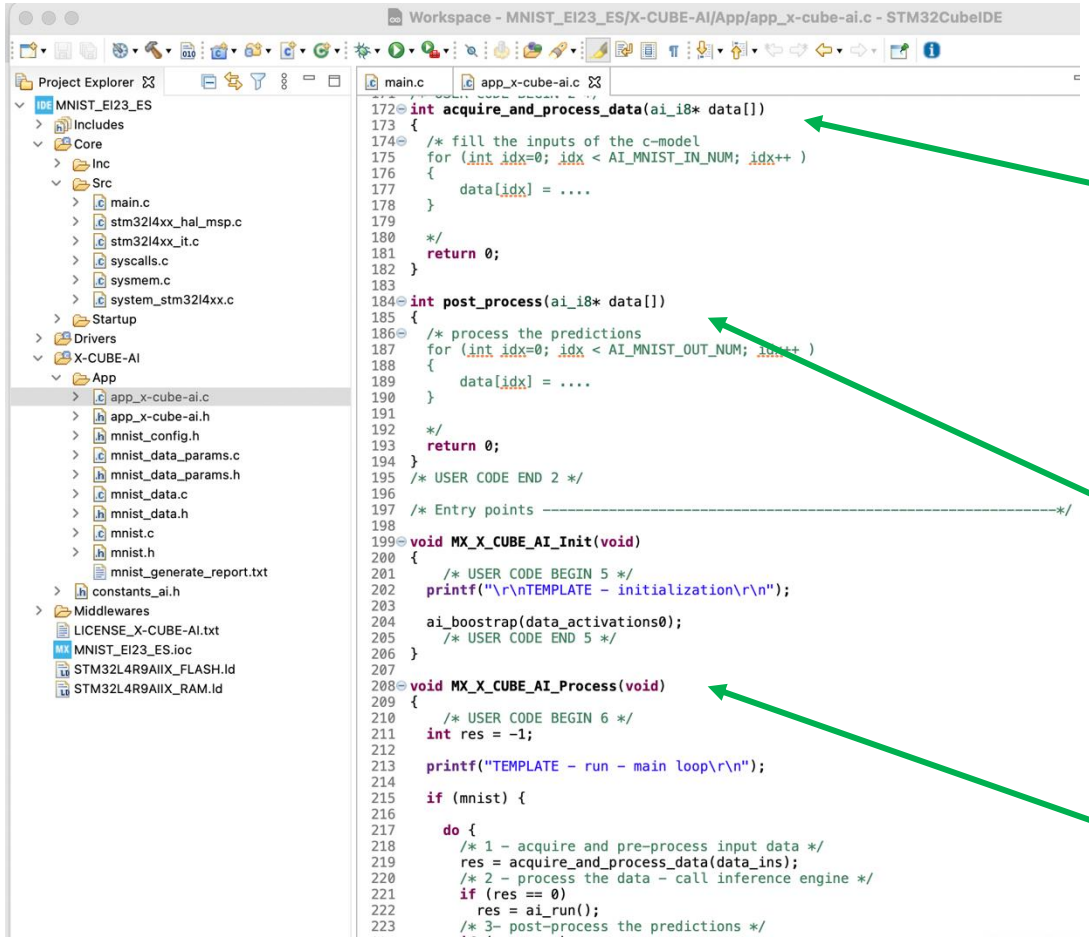
# Infering a DNN example

## Generate software code



# Infering a DNN example

## Generate software code / design the application in IDE…




**Generated functions:**

- pre-process function to manage input data
- AI_run post-process function to get output classification
- Inference process

# TODO

## MNIST example / Available files on Campus…

- Embedded IA presentation
- MNIST Dataset (mnist.npz)
- Validation Dataset
- Python files
  - CNN definition
  - STM32 – UART communication
- STM32L4R9 Discovery Board Datasheet




# Merci de votre attention

INSTITUT
Mines-Télécom

Liberté • Égalité • Fraternité
RÉPUBLIQUE FRANÇAISE
MINISTÈRE DE L'ÉCONOMIE
DE L'INDUSTRIE ET DU NUMÉRIQUE

INSPIRING
INNOVATION
SINCE 1816